LAB ASSIGNMENT 15.2

Lab 15 – Backend API Development: Creating RESTful Services with AI

Name: D.POORNANDA CHALAM **Roll No:** 2503A52361 **Batch:** 13

Task 1 – Setup FastAPI Backend

Question / Objective:
Setup FastAPI Backend with a root GET endpoint.

AI Prompt:
"Create a basic FastAPI app with a root GET endpoint that returns a welcome message."

Source Code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    """Return a welcome message."""
    return {"message": "Welcome to AI-powered FastAPI"}
```

Terminal Output (VS Code Execution):

```
[Running] uvicorn main:app --reload --port 8000
INFO:     Started server process [14820]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

HTTP Request Verification:
GET http://127.0.0.1:8000/
Response Status: 200 OK
Response Body:
{
  "message": "Welcome to AI-powered FastAPI"
}
```

Justification & Technical Explanation:
FastAPI creates the ASGI backend web application. The root GET endpoint at '/' acts as a health check and welcome entry point, returning a structured JSON payload confirming that the server is operational.

Task 2 – API – Create and Read Items

Question / Objective:
Add GET and POST endpoints to retrieve and create items using an in-memory Python list.

AI Prompt:
"Add GET and POST endpoints to retrieve and create items using a Python list."

Source Code:

```
items = []

@app.get("/items")
def get_items():
    """Return all stored items."""
    return items

@app.post("/items")
def add_item(item: dict):
    """Add an item to the list."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

Terminal Output (VS Code Execution):

```
GET /items:
Request: GET http://127.0.0.1:8000/items
Response: 200 OK
Body: []

POST /items:
Request: POST http://127.0.0.1:8000/items
Payload: {"name": "Pen", "price": 20}
Response: 200 OK
Body:
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```

Justification & Technical Explanation:
The GET endpoint queries and returns all stored item objects serialized into JSON. The POST endpoint accepts a JSON request body, appends the newly created record to the in-memory items list, and returns a 200 confirmation receipt.

Task 3 – Update Existing Item

Question / Objective:
Create a PUT endpoint that updates an item by its zero-based index and reports when the item is not found.

AI Prompt:
"Create a PUT endpoint that updates an item by its zero-based index and reports when the item is not found."

Source Code:

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """Update an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Terminal Output (VS Code Execution):

```
PUT /items/0:
Request: PUT http://127.0.0.1:8000/items/0
Payload: {"name": "Pencil", "price": 10}
Response Status: 200 OK
Response Body:
{
  "message": "Item updated",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

Justification & Technical Explanation:
The PUT endpoint accepts a path parameter `index` and new data payload. It validates that the index is within range [0, len(items) - 1]. If out-of-bounds, it raises an HTTPException with HTTP status code 404 ('Item not found'); otherwise it performs an atomic in-place update.

Task 4 – Delete Item

Question / Objective:
Create a DELETE endpoint that removes an item by its index and returns the deleted item.

AI Prompt:
"Create a DELETE endpoint that removes an item by its index and returns the deleted item."

Source Code:

```
@app.delete("/items/{index}")
def delete_item(index: int):
    """Delete an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Terminal Output (VS Code Execution):

```
DELETE /items/0:
Request: DELETE http://127.0.0.1:8000/items/0
Response Status: 200 OK
Response Body:
{
  "message": "Item removed",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}

DELETE /items/0 (Subsequent attempt):
Response Status: 404 Not Found
Response Body:
{
  "detail": "Item not found"
}
```

Justification & Technical Explanation:
The DELETE endpoint removes the target item using `pop(index)`. If the target item has already been removed or the supplied index is invalid, it safely produces a 404 Not Found HTTP response error.

Task 5 – Add Auto-Generated Documentation

Question / Objective:
Add comments, docstrings, endpoint summaries, and Swagger documentation to the FastAPI endpoints.

AI Prompt:
"Add comments, docstrings, endpoint summaries, and Swagger documentation to the FastAPI endpoints."

Source Code:

```python
from fastapi import FastAPI, HTTPException

app = FastAPI(
    title="Items CRUD API",
    description="API for creating, reading, updating, and deleting items",
    version="1.0.0"
)

items = []

@app.get("/items", summary="Get all items")
def get_items():
    """Return all items stored in memory."""
    return items

@app.post("/items", summary="Create an item")
def add_item(item: dict):
    """Create and return a new item."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

```python
@app.put("/items/{index}", summary="Update an item")
def update_item(index: int, item: dict):
    """Update an existing item by index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}", summary="Delete an item")
def delete_item(index: int):
    """Delete an item by index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Terminal Output (VS Code Execution):

```
Interactive OpenAPI Swagger Documentation:
URL: http://127.0.0.1:8000/docs
UI Title: Items CRUD API v1.0.0
Description: API for creating, reading, updating, and deleting items

Documented Endpoints:
  GET    /items          Get all items
  POST   /items          Create an item
  PUT    /items/{index}  Update an item
  DELETE /items/{index}  Delete an item
```

Justification & Technical Explanation:

FastAPI uses Python type hints and docstrings to generate an interactive OpenAPI (Swagger) UI at `/docs`. This allows developers to inspect request schemas, parameter types, HTTP status codes, and test endpoints interactively without external tools like Postman.